

Sistema Automatizado de Gestión de Rutinas Deportivas (SAGERD)

S. Hidalgo Gonzalez¹ y V. Lagos Torres²

¹ Ingeniería Civil Informática. Departamento de ciencias de la computación e informática,
Facultad de Ingeniería, Universidad Andres Bello, Viña del mar.

² Ingeniería Civil Informática. Departamento de ciencias de la computación e informática,
Facultad de Ingeniería, Universidad Andres Bello, Viña del mar.

Asignatura: PTEC102 – Paradigmas de programación - NRC 13046

Docente: Juan Calderon Maureira

Fecha de entrega: 16 de abril de 2026

RUT: 21.876.861-k y 21.925.820-8



Introducción

La planificación deportiva es una herramienta fundamental en las ciencias del ejercicio para estructurar de manera progresiva y segura los estímulos que un atleta recibe, moviéndose desde niveles de intensidad básicos hasta el alto rendimiento. Sin embargo, como se observa en la gestión diaria de centros deportivos, aplicar esto de forma personalizada es difícil porque en ocasiones no hay un registro claro de la carga previa o cuesta equilibrar el tiempo total de las sesiones con los objetivos físicos del cliente. Esta tarea nace de la motivación de crear una herramienta práctica para que un deportista pueda armar rutinas de entrenamiento de forma ordenada, asegurando que cada ejercicio seleccionado cumpla realmente con el nivel de exigencia que se busca trabajar.

El objetivo fundamental de este trabajo es desarrollar un programa en C++ que funcione como un sistema de gestión de rutinas inteligentes. El sistema permite gestionar un catálogo de ejercicios cardiovasculares y de fuerza, cada uno con una descripción detallada de su ejecución para garantizar la seguridad del usuario y facilitar el seguimiento de su progreso. Además, el software es capaz de buscar ejercicios según su nivel de intensidad, evitar la repetición de movimientos en semanas consecutivas y calcular automáticamente el tiempo estimado total que le tomará al cliente completar su entrenamiento. Para lograr esto, el desarrollo se enfoca en implementar un código funcional bajo el paradigma de la programación orientada a objetos, ejecutable directamente desde la consola de comandos, facilitando un uso rápido y eficiente para el usuario final.

El presente documento se estructura de la siguiente manera, en primer lugar, se describe la solución propuesta, detallando las clases diseñadas, las relaciones entre ellas y las decisiones de diseño tomadas, acompañadas del diagrama de clases correspondiente. Finalmente, se presentan las conclusiones del trabajo, con una reflexión sobre los objetivos alcanzados y los aprendizajes obtenidos durante el desarrollo.

Descripción de la solución

La arquitectura del sistema propuesto se fundamenta en un modelo de gestión centralizado que separa la definición de entidades de la lógica de la interfaz del usuario. Para lograr el nivel de modularidad adoptamos el paradigma de la programación orientada a objetos, utilizando polimorfismo como el mecanismo principal para gestionar la diversidad a las actividades físicas. Esta decisión arquitectónica nos permite que el núcleo del programa interactúe con diferentes tipos de ejercicios bajo una interfaz abstracta y unificada. Con esto el sistema gana una alta estabilidad por si en el futuro el centro deportivo deseara incorporar nuevas disciplinas como crossfit o aqua aerobio, bastaría con añadir nuevas clases al modelo sin necesidad de reescribir la lógica central de generación de rutinas o los menús de la consola.

Para llevar a cabo este modelo conceptual, el sistema lo estructuramos en torno a un ecosistema de cuatro clases interconectadas. En la cúspide de la jerarquía se define la clase Ejercicio, diseñada para encapsular el estado y comportamiento universal de cualquier actividad física. Esta superclase administra atributos críticos como el identificador único (código), el nombre, una descripción física, nivel de intensidad (un indicador de ejecución previa para el control de la sobrecarga) y el tiempo estimado en minutos. A partir de esta superclase derivan dos clases especializadas: Fuerza y Cardio, que representan tipos de ejercicios. Ambas subclases heredan de buena forma la estructura de su predecesora, pero implementan comportamientos específicos mediante la sobreescritura de métodos virtuales, permitiendo al sistema identificar de forma única la naturaleza del esfuerzo físico a través del método `getTipo()` y personalizar la salida de datos en pantalla.

En paralelo a la jerarquía de ejercicios se diseñó la clase Rutina, cuya responsabilidad es orquestrar la planificación del entrenamiento para un usuario específico. Esta clase actúa como un registro integral que guarda el nombre del cliente, el nivel de intensidad, y lo más importante: una colección dinámica de los ejercicios asignados. La clase Rutina no solo almacena datos, también provee un abanico de métodos propios, como `agregarEjercicio()`, `eliminarEjercicio()`, y un algoritmo iterativo en `calcularTiempoTotal()`, el cual recorre la colección de actividades sumando sus duraciones individuales para entregar una estimación temporal precisa de la sesión completa.

Las interacciones entre estas entidades fueron moldeadas a través de dos relaciones que reflejan la lógica del mundo real. Por una parte la relación de herencia entre Ejercicio y sus subclases garantiza la reutilización de código y el establecimiento de un contrato común para todas las actividades. Por otro lado, se estableció una relación de agregación entre la clase Rutina y la clase Ejercicio, materializada a través de un vector de punteros (`vector<Ejercicio*>`). Esta estructura de datos propone que una rutina se compone con múltiples ejercicios referenciados desde un catálogo principal, el cual permite al sistema manipular con mayor eficiencia las direcciones de memorias de los objetos sin la necesidad de duplicar la información de cada ejercicio cada vez que se asigna un cliente.

Para complementar la arquitectura y asegurar el rendimiento óptimo del software, se tomaron decisiones de diseño específicas que garantizan la estabilidad y escalabilidad del sistema. Una decisión fundamental fue el uso de punteros dentro del vector (`std::vector<Ejercicio*>`) en lugar de almacenar objetos por valor. Al trabajar con herencia, almacenar objetos derivados directamente en un contenedor de la clase base provoca un problema conocido como rebanado de objetos ("Object Slicing"), donde se pierden los atributos y métodos propios de las subclases alojadas en la memoria estática (stack). El uso de punteros asignados dinámicamente en el montículo (heap) mediante el operador `new` asegura que se mantenga la identidad real del objeto, permitiendo que el polimorfismo funcione correctamente gracias a la tabla virtual (vtable), la cual resuelve el enlazado dinámico en tiempo de ejecución para métodos como `getTipo()`. En consecuencia, debido a que el sistema utiliza memoria dinámica, se implementó un destructor virtual (`virtual ~Ejercicio()`) en la clase base. Esta decisión asegura que, al eliminar un ejercicio a través de un puntero genérico con el operador `delete`,

se llame primero al destructor de la clase derivada y luego al de la base, previniendo fugas de memoria críticas. Además, se incluyeron métodos iterativos explícitos como `liberarMemoria()` para recorrer y limpiar de manera segura las direcciones de memoria antes de finalizar la ejecución.

A continuación, en la Figura 1, se presenta el diagrama de clases relacionado al modelo descrito previamente.

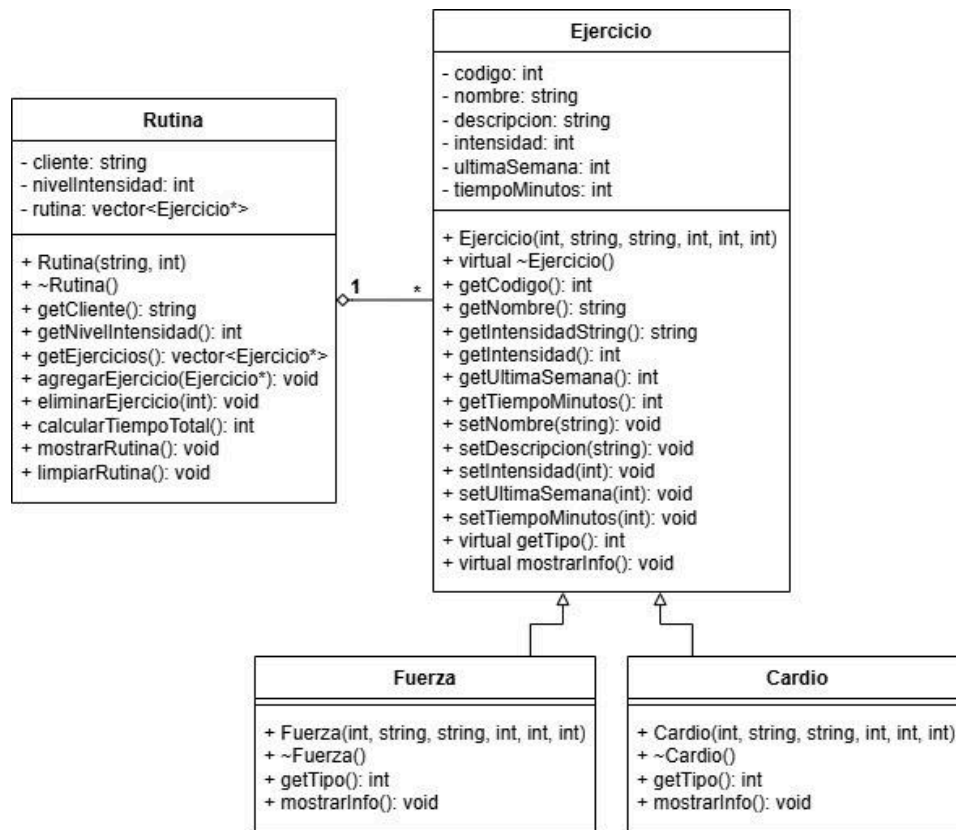


Figura 1. Diagrama de clases.

En el sistema, al generar una rutina, se excluyen de manera deliberada ejercicios realizados recientemente para evitar el estancamiento físico del usuario, en conjunto aplica un descarte aleatorio mediante la función `rand()` cuando el catálogo supera la cantidad solicitada, complementando la variabilidad a la planificación. Sin embargo para garantizar el mantenimiento de software optamos por una separación entre la definición de las interfaces (.h) y su implementación (.cpp), puesto que esta modularidad protege el encapsulamiento de datos, optimizando los tiempos de compilación y se complementa con funciones de validación robustas. Estas funciones de control de entrada fueron diseñadas específicamente para gestionar los buffers de lectura `std::cin` (utilizando el `getline` y el `cin.ignore`), lo cual blindo al programa ante tipos de datos erróneos ingresados por el usuario y asegura que el sistema no entre en un bucle infinito en la consola de comandos.

Debido a todo esto tomamos una decisión de bajo nivel altamente importante para el rendimiento del programa: la manipulación de las estructuras de datos mediante el paso de

referencia. Al suministrar el catálogo principal a funciones transaccionales, como `ingresarEjercicio`, `actualizarEjercicio` o `eliminarEjercicio`, en los cuales utilizamos la referencia vector<Ejercicio*>&, el cual hace que el sistema evite la sobrecarga creando copias profundas del arreglo en cada llamado a la pila de ejecución. Esto garantiza que cualquier alteración del catálogo persista globalmente con un coste operativo mínimo. De esta forma análogo, el control de flujo principal alojado en `gestionDeRutinas()` fue realizado a través de un máquina de estados simples, basados en la estructura cíclica del `do-while` combinada con un bloque condicional `switch`. Esto asegura el enrutamiento de opciones directo y garantiza que el ciclo de vida de la interfaz se mantenga activo de manera interactiva hasta que el usuario decida cerrar sesión, otorgando así una alta fluidez de experiencia de uso de terminal

Conclusión

El desarrollo de este proyecto nos permitió cumplir el objetivo propuesto de esta tarea, como la creación de un sistema inteligente, robusto y eficiente en C++ para la gestión de rutinas de entrenamiento. Por medio de la implementación rigurosa del paradigma de la programación orientada a objetos, logramos materializar la solución que no solo automatiza la planificación deportiva, sino que también respeta los principios de la variabilidad del estímulo y el control de la fatiga mediante evaluación de cargas previas.

Desde una perspectiva técnica, este trabajo nos representó un desafío significativo en el cual consolidamos conocimientos de bajo nivel, los que son vitales en la integración de software. La transición de entender la teoría del polimorfismo a aplicarlo para resolver el problema en la memoria dinámica, marcó un punto sin retorno en nuestro aprendizaje. Comprender cómo la máquina interactúa, la importancia de los destructores virtuales para evitar fugas de memoria y el impacto en el rendimiento al utilizar el paso de parámetros por referencia, nos dejó de evidencia que la calidad de un programa no solo reside únicamente en que la consola arroje el resultado esperado, sino también en la eficiencia, seguridad y escalabilidad de su arquitectura interna.

Sumado a esto, la creación de un modelo de clases a partir de un enunciado escrito en un lenguaje natural también simbolizó un reto, ya que fue necesario aplicar habilidades de abstracción para poder representar los conceptos solicitados como objetos, atributos y métodos, consolidando así conocimientos de alto nivel.

Finalmente, con la sólida base arquitectónica construida, separada en interfaces e implementaciones, asegura que el sistema no sea un producto estático. Gracias a la alta cohesión y bajo acoplamientos logrados, el programa está preparado para en un futuro sea escalado, ya sea integrando persistencia de datos mediante bases de datos, incorporando nuevas disciplinas al catálogo o migrando hacia una interfaz gráfica. Este proyecto en definitiva trasciende a más que solo la entrega de líneas de código, sino que también se consolida como una herramienta práctica que une de manera exitosa la ciencia de la computación con las ciencias del ejercicio.